

Day 8 - Facilitation Guide

(Advanced Python - Inheritance-MethodOverriding)

INDEX

- I. Recap of the previous session
- II. Inheritance Basics
- III. Overriding Methods
- IV. Using `super()`
- V. Multiple Inheritance

(1.5 hrs) ILT

I. Recap of the previous session

In the last session, we taught you many important concepts of static methods, class methods and Method overloading in Python. Here's a brief introductory recap session covering the key concepts you've taught in the last session:

Method Overloading:

- Method overloading refers to the ability to define multiple methods with the same name but different parameters within the same class.
- In Python, method overloading is not supported in the traditional sense. Python does not allow defining multiple methods with the same name but different parameter lists within a class.

Static Methods:

- A static method in Python is a method that belongs to a class rather than to an instance of the class.
- They are defined using the `@staticmethod` decorator and can be called on the class itself, without creating an instance of the class.

Class Methods: Class methods are methods that are bound to the class and not the instance of the class. They have access to the class itself rather than the individual objects. Class methods are defined using the `@classmethod` decorator and take `cls` as their first parameter, which refers to the class that is being used.

By the end of this session, students will grasp the fundamentals of inheritance in Python, including how to create and use base and derived classes to promote code reuse and extension. They will learn to override methods in derived classes to customize or enhance inherited behavior, and effectively use the `super()` function to access and manage methods from parent classes. Additionally, students will explore multiple inheritance, understanding its advantages and complexities, and how to apply it to create classes that inherit from multiple base classes while managing method resolution order.

Instance variables vs Class variables:

Feature	Instance Variables	Class Variables
Declaration	Inside the constructor (<code>__init__</code> method)	Inside the class, outside any methods
Scope	Specific to each object (instance)	Shared among all instances of the class
Access	Accessed using <code>self.variable_name</code>	Accessed using <code>ClassName.variable_name</code> or <code>self.variable_name</code>
Change	Changes affect only the specific instance	Changes affect all instances of the class

II. Inheritance Basics

Inheritance in Python is a key concept in object-oriented programming (OOP) that allows a class (called the child or derived class) to inherit attributes and methods from another class (called the parent or base class). This promotes code reusability and helps in creating a hierarchical relationship between classes.

Basic Concepts of Inheritance

- **Parent (Base) Class:** The class whose properties and methods are inherited by another class.
- **Child (Derived) Class:** The class that inherits properties and methods from the parent class. The child class can also have its own properties and methods.
- **super() Function:** The `super()` function is used to call the methods of the parent class from the child class. It's particularly useful when you want to extend the functionality of the parent class method in the child class.

Types of Inheritance:

- **Single Inheritance:** A child class inherits from one parent class.
- **Multiple Inheritance:** A child class inherits from more than one parent class.
- **Multilevel Inheritance:** A child class inherits from a parent class, and that parent class is itself a child of another class.
- **Hierarchical Inheritance:** Multiple child classes inherit from a single parent class.
- **Hybrid Inheritance:** A combination of multiple types of inheritance.

Syntax of Inheritance

class ParentClass:

```
# Parent class attributes and methods

pass

class ChildClass(ParentClass):

    # Child class inherits from ParentClass

    Pass
```

Example:

```
# Parent class
class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        return f"{self.name} makes a sound"

# Child class inheriting from Animal
class Dog(Animal):
    def speak(self):
        return f"{self.name} barks"

# Creating an instance of the child class
dog = Dog("Buddy")
print(dog.speak()) # Output: Buddy barks
```

Explanation:

- Animal is the parent class with an `__init__` method that initializes the name attribute and a `speak` method.
- Dog is the child class that inherits from Animal. It overrides the `speak` method to provide a specific implementation for dogs.
- When we create an instance of Dog and call the `speak` method, it uses the overridden method in the Dog class.

III. Overriding methods: In Python it is a feature that allows a child class to provide a specific implementation of a method that is already defined in its parent class. When a method is overridden, the method in the child class is called instead of the one in the parent class when an object of the child class invokes the method. This is useful for altering or extending the behavior of methods inherited from the parent class.

Key Points of Method Overriding:

Same Method Signature: The method in the child class must have the same name and parameters as the method in the parent class that it is overriding.

Polymorphism: Method overriding is a key feature of polymorphism in object-oriented programming, where a child class can redefine methods and the appropriate method is called based on the object type.

Syntax of Method Overriding:

```
class ParentClass:

    def some_method(self):

        # Parent class method

        print("This is the parent class method.")

class ChildClass(ParentClass):

    def some_method(self):

        # Overriding the parent class method

        print("This is the child class method.")
```

Example:

```
# Parent class
class Animal:
    def speak(self):
        return "Animal makes a sound"

# Child class
class Dog(Animal):
    def speak(self):
        return "Dog barks"

# Child class
class Cat(Animal):
    def speak(self):
        return "Cat meows"

# Creating instances of the child classes
dog = Dog()
cat = Cat()

print(dog.speak()) # Output: Dog barks
```

```
print(cat.speak()) # Output: Cat meows
```

Explanation:

- Animal is the parent class with a speak method that returns a generic message.
- Dog and Cat are child classes that override the speak method of the parent class to return specific messages.
- When the speak method is called on an instance of Dog or Cat, the overridden method in the respective class is executed.

Output:

```
Dog barks
Cat meows
```

Practical Use Cases of Method Overriding

Customizing Behavior in Subclasses: In a large application, you might have a general method in the base class that works for most subclasses but requires customization for specific cases.

Enhancing Functionality: A subclass can enhance or add to the functionality provided by the base class by overriding methods.

Ensuring Consistent API: Overriding allows subclasses to provide their specific implementations while maintaining a consistent method API across all classes in the hierarchy.

Example:

```
# Parent class
class Account:
    def get_interest_rate(self):
        return 0.0

# Child class for savings account
class SavingsAccount(Account):
    def get_interest_rate(self):
        return 4.0

# Child class for fixed deposit account
class FixedDepositAccount(Account):
    def get_interest_rate(self):
        return 7.5

# Creating instances
```

```
savings = SavingsAccount()
fixed_deposit = FixedDepositAccount()

print(savings.get_interest_rate()) # Output: 4.0
print(fixed_deposit.get_interest_rate()) # Output: 7.5
```

Explanation:

- The Account class has a method `get_interest_rate()` that returns 0.0 by default.
- SavingsAccount and FixedDepositAccount override this method to provide specific interest rates.
- When calling `get_interest_rate()` on instances of these subclasses, the overridden methods are executed.

Output:

4.0
7.5

IV. Using super()

The `super()` function in Python is used to call a method from the parent class inside a child class. It's particularly useful when you want to extend or modify the behavior of inherited methods without completely overriding them. By using `super()`, you can invoke the parent class's methods, allowing the child class to build upon the functionality of the parent class.

Key Points About super()

Access Parent Class Methods: `super()` allows you to call methods from the parent class within the child class, enabling code reuse and extension.

Multiple Inheritance: In the case of multiple inheritance, `super()` follows the method resolution order (MRO) to determine the order in which classes are accessed.

Avoiding Hard-Coding: `super()` dynamically finds the parent class, which helps avoid hard-coding the parent class name, making the code more maintainable and flexible.

Basic Syntax:

```
class ParentClass:
    def some_method(self):
        print("This is the parent class method.")

class ChildClass(ParentClass):
```

```
def some_method(self):
    super().some_method() # Call the parent class method
    print("This is the child class method.")
```

Example:

```
# Parent class
class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        return f"{self.name} makes a sound"

# Child class
class Dog(Animal):
    def __init__(self, name, breed):
        super().__init__(name) # Call the parent class's __init__ method
        self.breed = breed

    def speak(self):
        parent_speak = super().speak() # Call the parent class's speak method
        return f"{parent_speak} and barks. Breed: {self.breed}"

# Creating an instance of the child class
dog = Dog("Buddy", "Golden Retriever")
print(dog.speak()) # Output: Buddy makes a sound and barks. Breed: Golden Retriever
```

Explanation:

- **Animal Class:** The Animal class has an `__init__` method that initializes the name attribute and a `speak` method that returns a generic message.
- **Dog Class:** The Dog class inherits from Animal. It overrides the `__init__` method and adds a new attribute `breed`. Inside the `__init__` method of Dog, `super().__init__(name)` is used to call the `__init__` method of the parent class, ensuring the name attribute is properly initialized.
- **Overriding speak Method:** The `speak` method in Dog calls the `speak` method from Animal using `super().speak()` and then extends its functionality by adding additional information about the dog's breed.

Output:

```
Buddy makes a sound and barks. Breed: Golden Retriever
```

Example: Imagine you have a base class `Vehicle` and you want to extend it for specific vehicle types like `Car` and `ElectricCar`. The `ElectricCar` needs to call the parent class methods and also perform additional initialization.

```
class Vehicle:
    def __init__(self, make, model):
        self.make = make
        self.model = model

    def start(self):
        return f"{self.make} {self.model} is starting."

class Car(Vehicle):
    def __init__(self, make, model, engine_type):
        super().__init__(make, model)
        self.engine_type = engine_type

    def start(self):
        return f"{super().start()} Engine type: {self.engine_type}."

class ElectricCar(Car):
    def __init__(self, make, model, battery_capacity):
        super().__init__(make, model, "Electric")
        self.battery_capacity = battery_capacity

    def start(self):
        return f"{super().start()} Battery capacity: {self.battery_capacity} kWh."

# Creating an instance of ElectricCar
tesla = ElectricCar("Tesla", "Model S", 100)
print(tesla.start())
# Output: Tesla Model S is starting. Engine type: Electric. Battery capacity: 100 kWh.
```

Explanation:

- **Vehicle Class:** Basic attributes and methods for any vehicle.
- **Car Class:** Inherits from `Vehicle` and adds an `engine_type` attribute. Uses `super()` to initialize the parent class.
- **ElectricCar Class:** Inherits from `Car` and assumes an electric engine type. Adds `battery_capacity` and uses `super()` to call methods from both `Car` and `Vehicle`.

Using `super()` in this way allows the `ElectricCar` class to correctly inherit and extend functionality from both `Vehicle` and `Car`, maintaining a clear and logical class hierarchy.

Output:

Tesla Model S is starting. Engine type: Electric. Battery capacity: 100 kwh.

IV. Multiple Inheritance

Multiple inheritance is a feature in Python where a class can inherit attributes and methods from more than one parent class. This allows for a class to be created that combines the functionality of multiple classes. While powerful, multiple inheritance can introduce complexity, especially when different parent classes have methods or attributes with the same name.

Key Concepts of Multiple Inheritance

Inheritance from Multiple Classes: A class can inherit from two or more parent classes, allowing it to combine their behaviors and properties.

Method Resolution Order (MRO): Python uses a specific order, known as Method Resolution Order (MRO), to determine which method to call when there are methods with the same name in multiple parent classes. The `super()` function plays a significant role in handling this.

Diamond Problem: A potential issue in multiple inheritance occurs when two parent classes themselves inherit from a common ancestor. This can create ambiguity in which method or attribute should be inherited. Python resolves this using MRO.

Syntax of Multiple Inheritance

```
class ParentClass1:
    # Parent class 1
    pass

class ParentClass2:
    # Parent class 2
    pass

class ChildClass(ParentClass1, ParentClass2):
    # Child class inheriting from both ParentClass1 and ParentClass2
    pass
```

Example:

```
class Car:
    def start(self):
        return "Car engine started"
```

```
def drive(self):
    return "Car is driving"

class Boat:
    def start(self):
        return "Boat engine started"

    def sail(self):
        return "Boat is sailing"

class HybridVehicle(Car, Boat):
    def start(self):
        car_start = super().start() # Resolves to Car's start method due to MRO
        boat_start = Boat.start(self) # Explicitly calling Boat's start method
        return f"{car_start} and {boat_start}"

# Creating an instance of HybridVehicle
hybrid = HybridVehicle()

print(hybrid.start()) # Output: Car engine started and Boat engine started
print(hybrid.drive()) # Output: Car is driving
print(hybrid.sail()) # Output: Boat is sailing
```

Explanation:

- **Car Class:** Contains methods to start and drive related to a car.
- **Boat Class:** Contains methods to start and sail related to a boat.
- **HybridVehicle Class:** Inherits from both Car and Boat, combining their functionalities.
 - The start method in HybridVehicle uses super() to call the start method from Car (as it appears first in the MRO) and explicitly calls Boat.start() to include the boat's start behavior.

Output:

```
Car engine started and Boat engine started
Car is driving
Boat is sailing
```

Exercise:

Use GPT to write a program:

1. Design and implement a simple online library management system using Python classes. Your solution should demonstrate the use of instance variables, class variables, method overloading (simulated), static methods, and class methods.
2. Design a simple online shopping platform using Python classes to manage products, categories, and user interactions. Implement instance variables, class variables, method overloading (simulated), static methods, and class methods in your solution.

After the ILT, the student has to do a 30 min live lab. Please refer to the Day 8 Lab doc in the LMS.